Rumo

Manual técnico do projeto

Bancada de eletrônica interativa com 32 placas e circuitos vistos como no KiCad, em 3D e no editor de PCB, 252 partes identificadas e 25 laboratórios com fiação, comandos, passos, simuladores e código. Português e inglês, tema claro e escuro e quatro paletas de cores, tudo no navegador.

Copyright (©) 2026 Erik Pimenta
Versão online: https://ravelroot.github.io/Circuit-Board-Bench-Lab/

Versão 1.2.0 · Novembro de 2026 · Todos os direitos reservados

Versão 1.2.0 · Novembro de 2026

Novidades da versão 1.2.0

A versão 1.2.0 traz a nova identidade visual do Rumo, no mesmo padrão do **Grafo**, do **Crivo** e do **Pluma**: logo preta em fundo branco, fontes do sistema, a paleta da casa como padrão, um **menu de paletas** com quatro opções, cada uma com tema claro e escuro, e uma **barra de status** no rodapé. As placas, as partes, os projetos de PCB e os laboratórios não mudaram, e a janela no estilo KiCad mantém as cores do programa.

> **Onde está:** tudo sobre a nova aparência fica nos capítulos "A interface" (barra superior, idiomas, temas e paletas, barra de status), "Arquitetura do código" (aparência) e "Personalização".

Área	O que entrou
Logo	Grade de pixels em preto num quadrado branco, na barra superior, no ícone do site e nas imagens do README
Fontes	`system-ui` e `ui-monospace` no lugar de Bricolage Grotesque, Instrument Sans e IBM Plex Mono; em produção resta só o three.js
Paletas	Rumo (linho, vinho, ardósia e palha, padrão), Bento, Sonda e Nexo, cada uma com tema claro e escuro, pelo botão de paleta ou pela tecla `C`
Tema	A primeira visita segue o tema do sistema; tema e paleta são aplicados antes da primeira pintura, sem piscar
Interface	Títulos, botões, chips de placa, abas dos labs e cartões no estilo dos outros projetos; destaque das partes na cor da paleta
Barra de status	Versão, placa ou lab, peças na bancada, labs concluídos, tema e paleta
Testes	Novos testes de ponta a ponta para o tema do sistema, o menu e o atalho de paletas e a barra de status

Sumário

Visão geral 5

- Números do catálogo 5
- Tecnologias 6

Instalação e execução 7

- Requisitos 7
- Passo a passo 7

Publicação no GitHub Pages 8

A interface 9

- Barra superior 9
- Catálogo 9
- Página da placa 9
- O que tem na sua bancada? 10
- Todos os laboratórios 11
- Idiomas, temas e paletas 11
- Barra de status 13

O visualizador 3D 14

O editor de PCB 16

Os laboratórios 18

Os 16 simuladores 19

Arquitetura do código 21

- Estado 21
- Aparência 21
- Rotas 21
- Bancada 21

O gerador de projetos de PCB 22

Como funciona o renderizador 3D 23

Formato de dados 24

- Placa 24
- Laboratório 25

Como adicionar 26

- Uma placa 26
- Um laboratório 26

Testes e segurança 27

- Testes e CI 27
- Segurança 27

Personalização 28

- Paletas de cores 28
- Tokens, fontes e cores do KiCad 28

Solução de problemas 29

Atalhos e licença 30

- Atalhos de teclado 30
- Licença 30

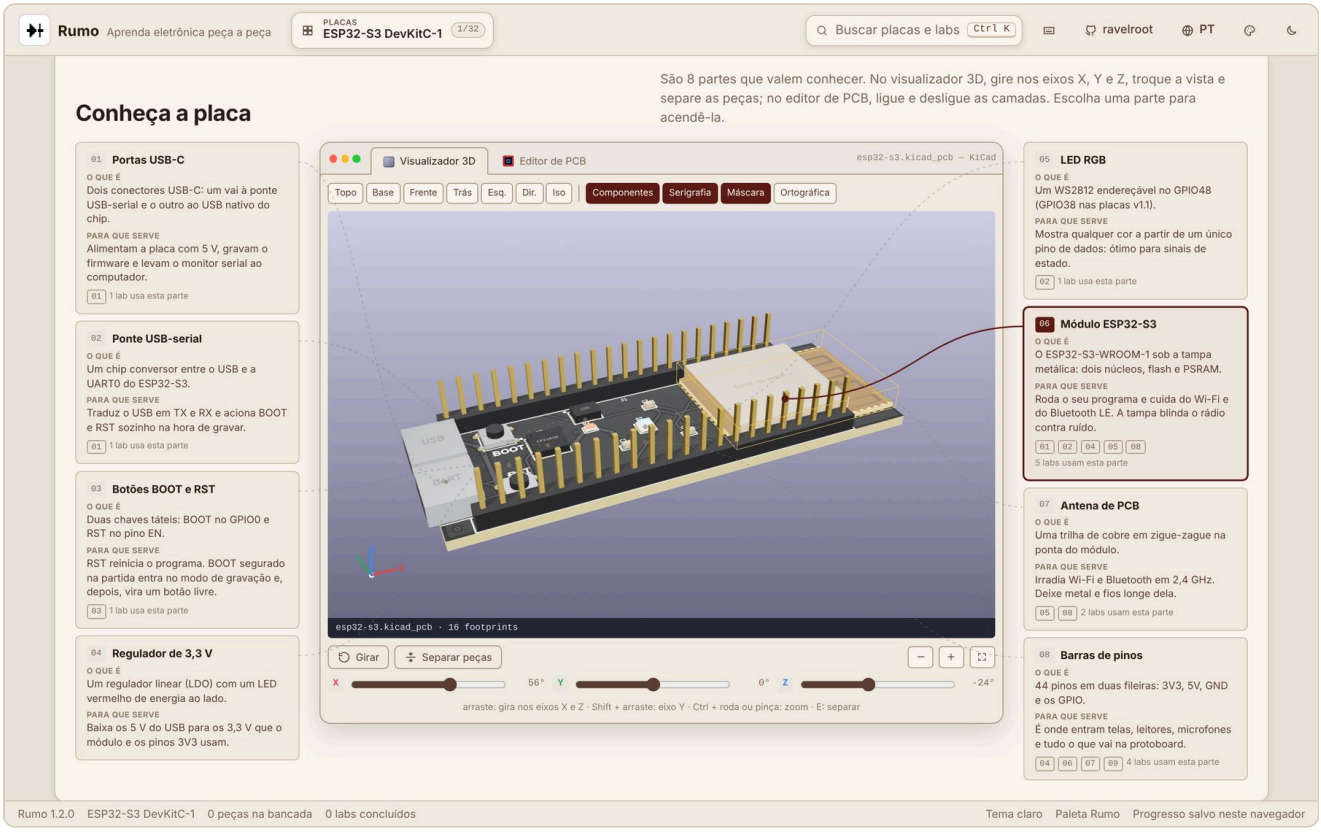
Catálogo completo 32

- As 32 placas e circuitos 32
- Os 25 laboratórios 34

Visão geral

O Rumo é uma aplicação web estática, escrita em TypeScript sem framework, que mostra placas de circuito como no KiCad, num visualizador 3D realista em WebGL e num editor de PCB técnico em 2D, e explica cada peça. Para as placas que têm laboratórios, ele também funciona como guia de bancada: inventário de peças, lista do que dá para montar, fiação animada, comandos, passos, simulador e código.

O projeto segue o mesmo padrão do Ocelo, e a partir da versão 1.2.0 também a mesma identidade visual do Grafo, do Crivo e do Pluma: Vite, TypeScript em modo estrito, interface montada pelo DOM, política de segurança de conteúdo no build, português e inglês, tema claro e escuro, quatro paletas de cores, fontes do sistema, Vitest, Playwright, GitHub Actions e GitHub Pages.



ESP32-S3 no visualizador 3D no estilo KiCad, com o módulo aceso e as oito partes identificadas ao redor.

Números do catálogo

Item	Quantidade
Placas e circuitos	32, em cinco categorias
Partes identificadas	252
Componentes desenhados	427
Footprints de PCB gerados	410, com mais de 12 mil pads, 1.020 trilhas e 677 vias
Laboratórios	25, com 16 simuladores

Tecnologias

Tecnologia	Papel
TypeScript 7 (modo estrito)	Toda a aplicação, os dados das placas e dos labs
three.js	Visualizador 3D em WebGL, a única dependência de produção
SVG e Canvas	Editor de PCB, fiação, miniaturas e simuladores
Web Audio	Som do sintetizador, gerado no navegador
Vite 8	Servidor de desenvolvimento e build estático
Vitest e Playwright	Testes unitários e de interface
Fontes do sistema	`system-ui` para o texto e `ui-monospace` para pinos, comandos e código

Instalação e execução

A licença permite baixar o Rumo e rodá-lo no próprio computador, sem alterações, para uso próprio. Modificar, vender, redistribuir ou hospedar em outro endereço depende de autorização por escrito do autor; as seções sobre código, personalização e como adicionar placas e labs servem ao autor e a quem tiver essa autorização.

Requisitos

- Node.js 22 ou superior (o arquivo `.nvmrc` fixa a versão) e npm.
- Um navegador atual com WebGL (Chrome, Edge, Firefox ou Safari).

Passo a passo

1. Clone o repositório e entre na pasta: `git clone https://github.com/ravelroot/Circuit-Board-Bench-Lab.git` e `cd Circuit-Board-Bench-Lab`.
2. Instale as dependências exatamente como estão no lockfile: `npm ci`. Na versão 1.2.0 as três fontes do Fontsource saíram do `package.json`, então o `npm ci` remove esses pacotes.
3. Rode em desenvolvimento: `npm run dev` e abra `http://localhost:5173`.
4. Gere o build de produção: `npm run build`. A pasta `dist/` é o site completo.
5. Confira o build: `npm run preview` e abra `http://localhost:4173`.

```
npm ci
npm run dev          # desenvolvimento com recarga
npm run typecheck    # TypeScript estrito
npm test             # testes unitários
npm run build        # dist/
npm run preview      # servidor do build
```

> **Caminhos relativos:** o `dist/` usa `base: './'`, então funciona em qualquer subpasta, inclusive no endereço do GitHub Pages. O `aparencia.js`, que aplica o tema e a paleta antes da primeira pintura, vai junto, copiado de `public/`.

Publicação no GitHub Pages

1. Envie o projeto para o repositório `ravelroot/Circuit-Board-Bench-Lab` na branch `main`.
2. No GitHub, abra **Settings** → **Pages**.
3. Em **Build and deployment**, escolha **GitHub Actions** como fonte.
4. Abra a aba **Actions** e acompanhe o workflow **GitHub Pages**: ele roda `npm ci`, os testes, o build e publica.
5. O site fica em `https://ravelroot.github.io/Circuit-Board-Bench-Lab/`. Cada novo push na `main` publica de novo.

O workflow **CI** roda em todo push e pull request: auditoria de dependências, TypeScript, testes unitários, build e testes de interface com Playwright. Se algo falhar, a publicação não acontece.

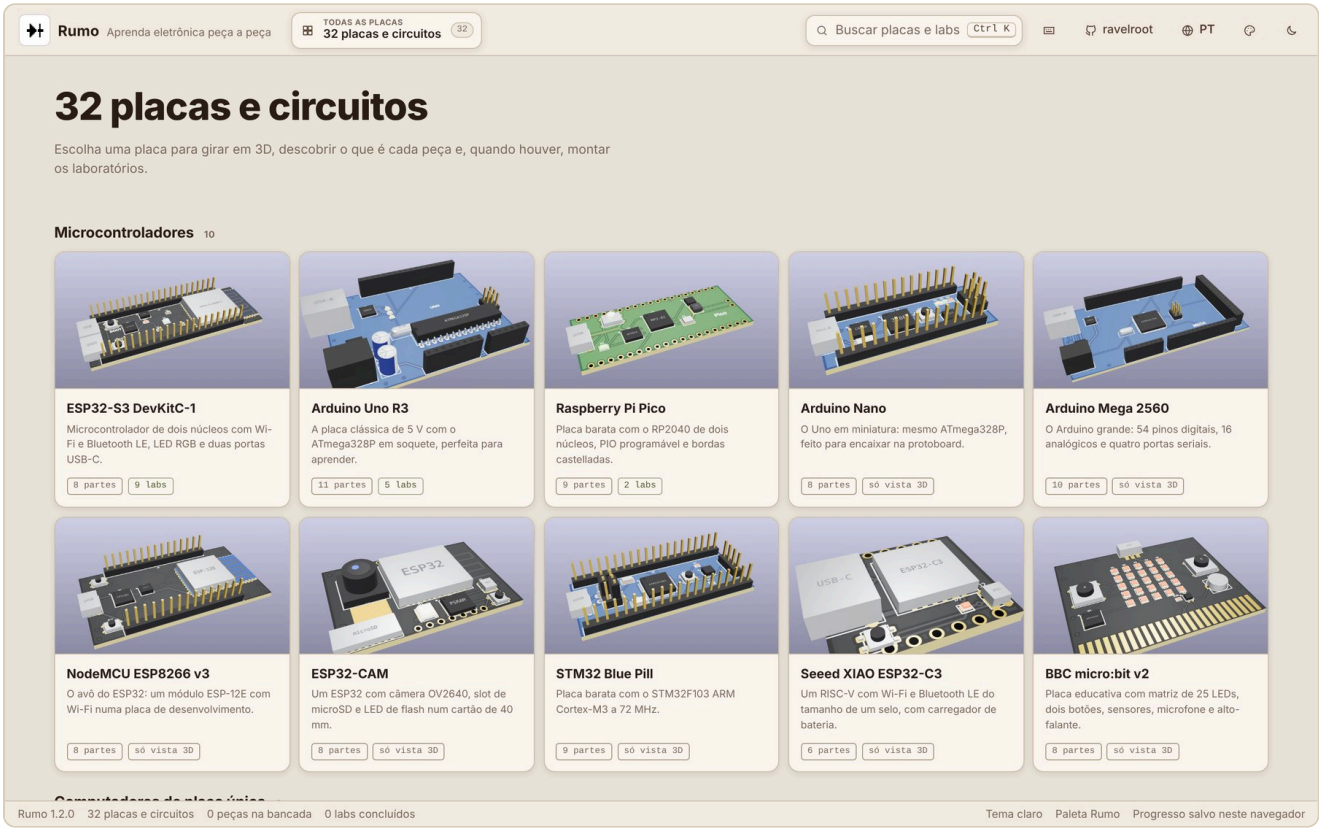
A interface

Barra superior

Segue o desenho do Grafo, do Crivo e do Pluma. Da esquerda para a direita: a logo, em preto num quadrado branco, com o nome e o lema (abre o catálogo); a placa atual com a posição no catálogo (por exemplo, 1/32); a busca (`Ctrl` + `K`); a folha de atalhos; o link do código-fonte; o idioma; o botão de paleta e o botão de tema. A barra fica fixa ao rolar. Os botões de ícone são discretos e ganham um fundo suave ao passar o mouse.

Catálogo

Em `#/placas`, as 32 placas aparecem em cinco grupos, cada uma com uma miniatura 3D renderizada no estilo do visualizador do KiCad (gerada só quando o cartão aparece na tela e guardada em memória), o resumo, o número de partes e o número de labs (ou "só vista 3D").

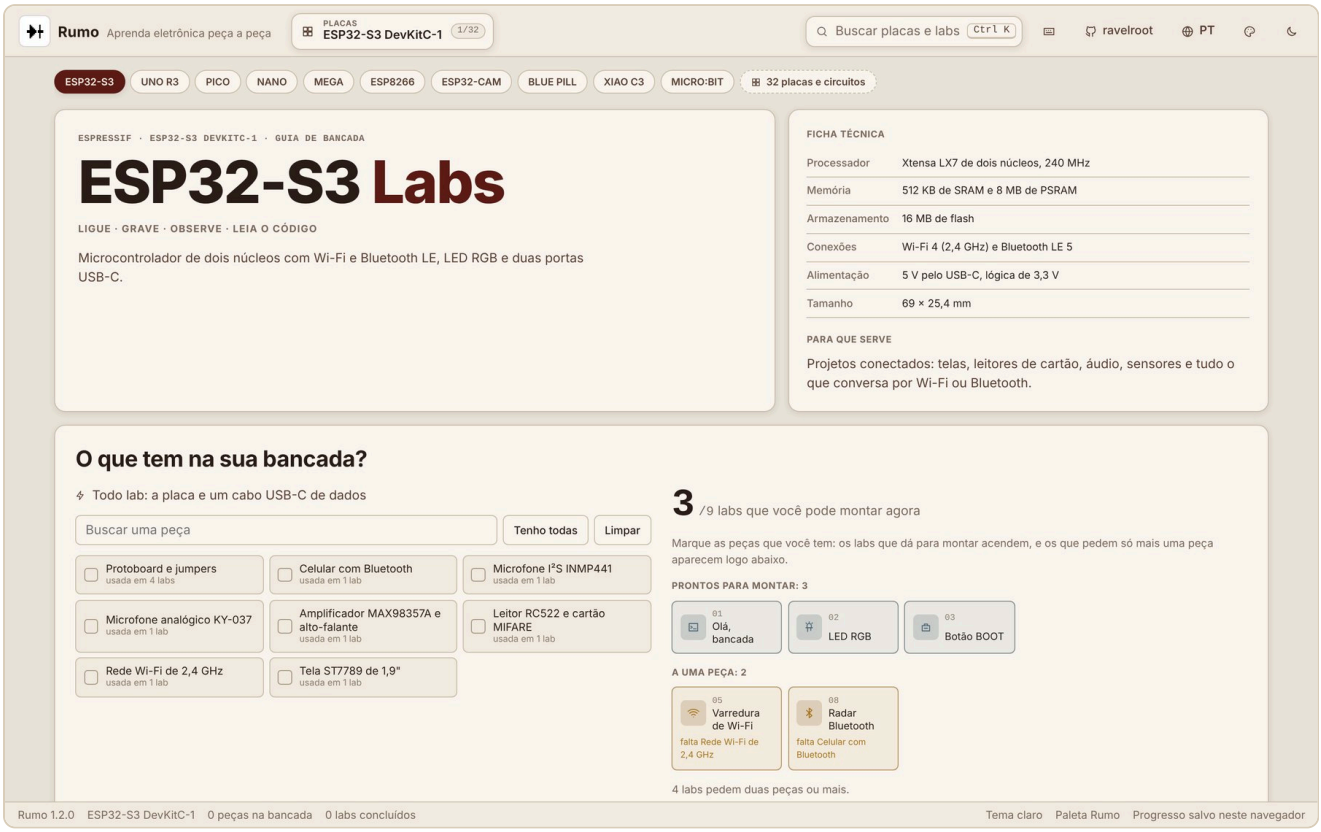


Catálogo com as placas renderizadas em 3D.

Página da placa

No topo, os chips trocam entre as placas da mesma categoria; o chip da placa aberta fica preenchido com a cor de destaque da paleta. O bloco principal mostra o fabricante, o nome, o título grande (com "Labs" na cor de destaque quando a placa tem laboratórios), os verbos do guia e o resumo; nas placas sem labs, um aviso diz que a placa tem só a

vista 3D. À direita ficam a ficha técnica e o texto "para que serve". O andamento dos labs aparece no mapa de laboratórios, pela cor de cada ladrilho.

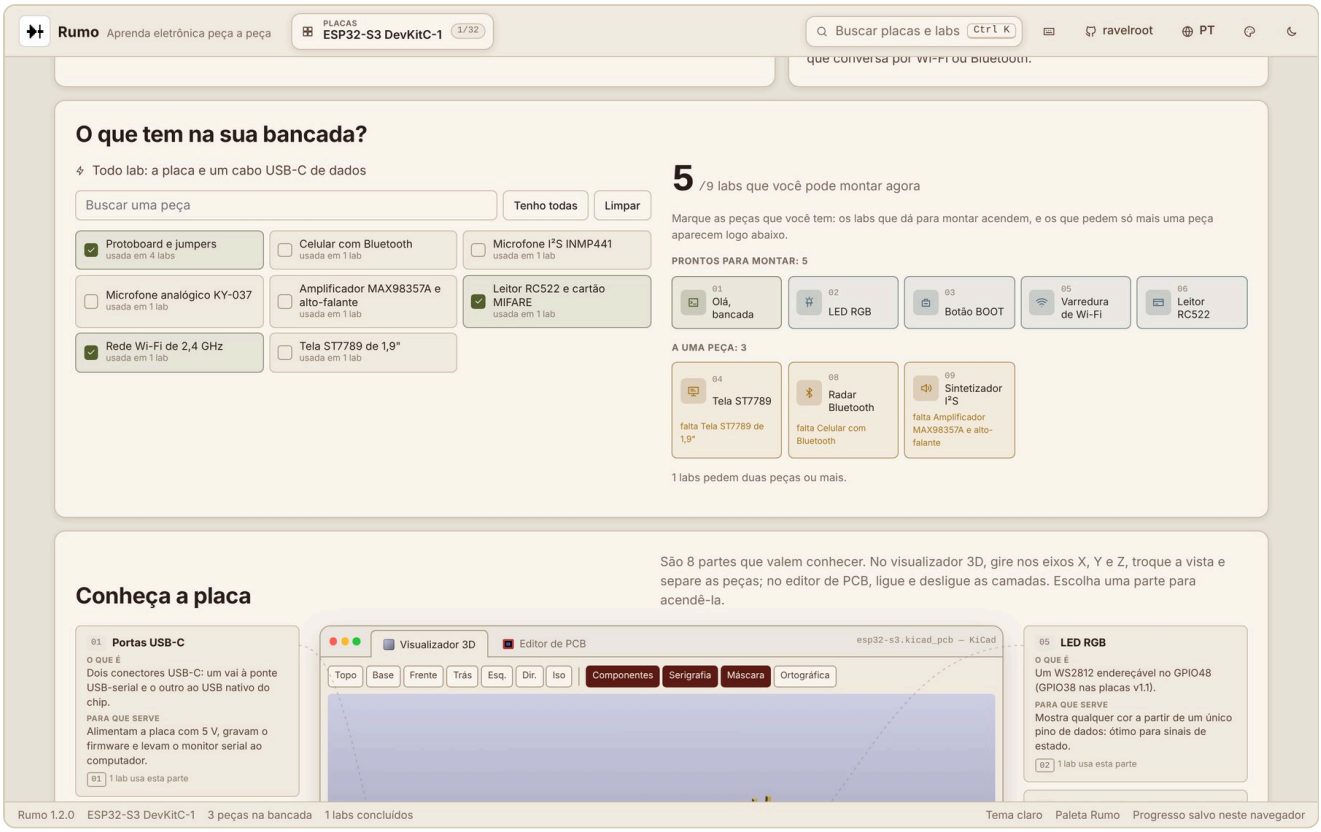


Página da ESP32-S3: troca de placa, título, resumo e ficha técnica.

O que tem na sua bancada?

Aparece nas placas com labs. A lista traz cada peça usada pelos labs e em quantos ela aparece. Marcar ou desmarcar atualiza na hora o lado direito: quantos labs dá para montar agora, os prontos, os que estão a uma peça (com o nome da

peça que falta) e quantos pedem duas ou mais. "Tenho todas" marca tudo daquela placa e "Limpar" desmarca. A busca filtra a lista.



Bancada com peças marcadas, labs prontos e labs a uma peça.

Todos os laboratórios

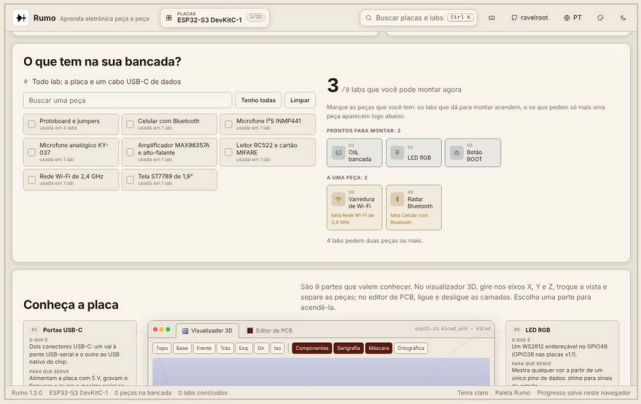
Um tema por linha, com um anel colorido, o resumo e quantos labs daquele tema estão ao alcance. Cada ladrilho tem o ícone, o número e o título, e a borda mostra o estado: oliva para feito por você, anil para pronto na bancada, ouro para falta uma peça e cinza para faltam peças. Os tons seguem a paleta e o tema escolhidos.

Idiomas, temas e paletas

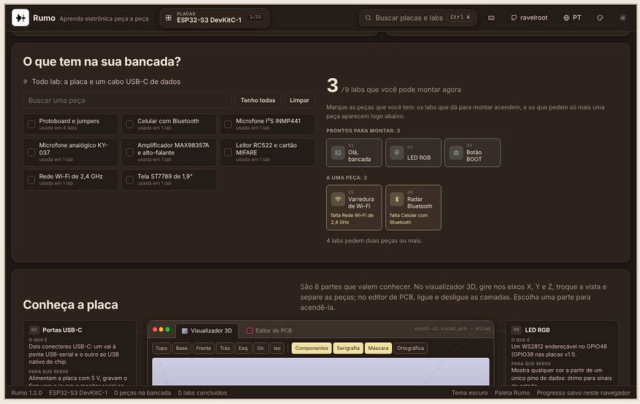
O botão com o globo alterna entre português e inglês; na primeira visita o Rumo segue o idioma do navegador. O botão de lua ou sol alterna entre os temas claro e escuro; na primeira visita o tema segue o do sistema. O botão com o ícone de paleta abre o menu das quatro paletas, cada uma com quatro amostras de cor: Rumo (padrão), Bento, Sonda e Nexo. A tecla `C` passa para a próxima.

A escolha fica salva no navegador e é aplicada antes de a tela aparecer, sem piscar. A janela no estilo KiCad mantém as cores do Pcbnew e o fundo do visualizador 3D em qualquer paleta, para continuar igual ao programa.

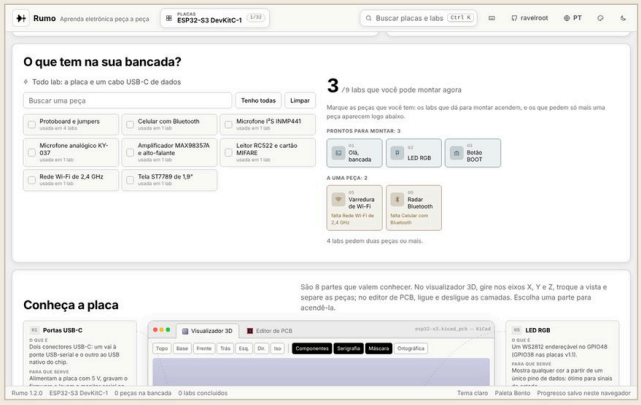
Rumo (padrão) · tema claro



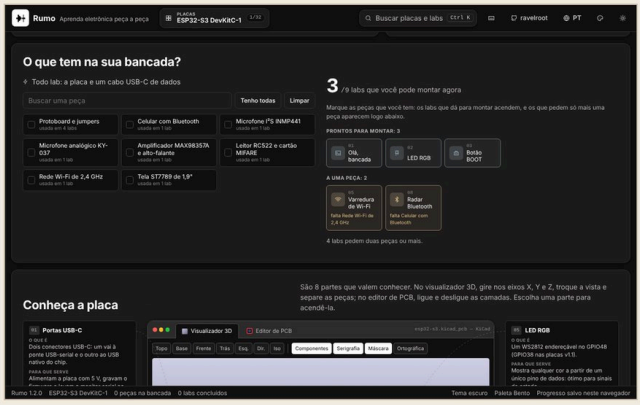
Rumo (padrão) · tema escuro



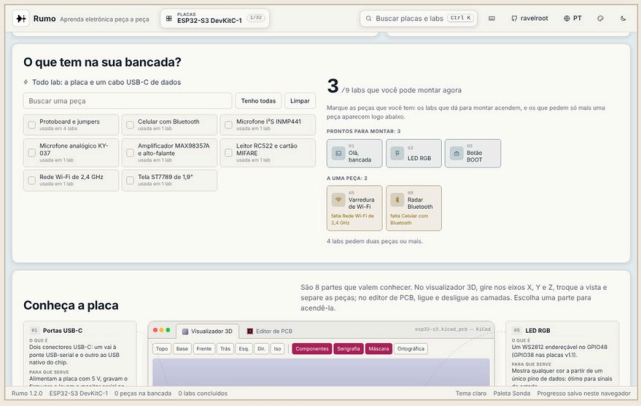
Bento · tema claro



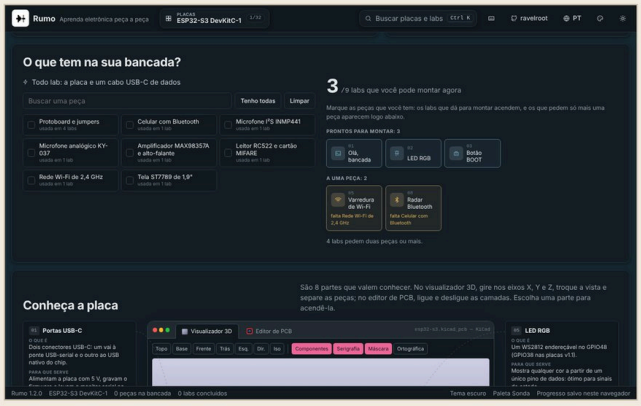
Bento · tema escuro



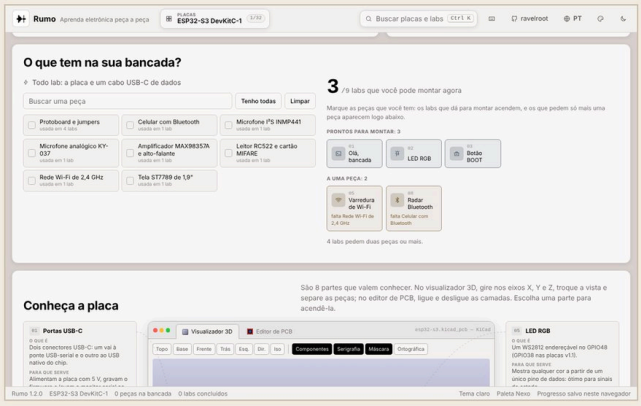
Sonda · tema claro



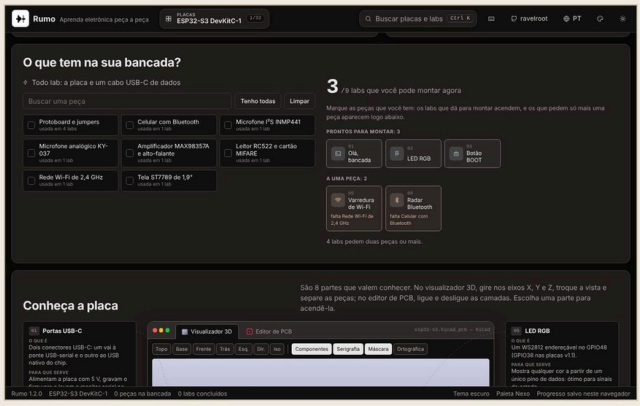
Sonda · tema escuro



Nexo · tema claro



Nexo · tema escuro



O Rumo nas quatro paletas, cada uma no tema claro e no escuro.

Barra de status

O rodapé mostra, à esquerda, a versão, a placa ou o lab abertos, quantas peças estão na bancada e quantos labs foram concluídos; à direita, o tema, a paleta e o aviso de que o progresso fica salvo neste navegador. A barra acompanha a rolagem; no celular, só o lado esquerdo aparece.

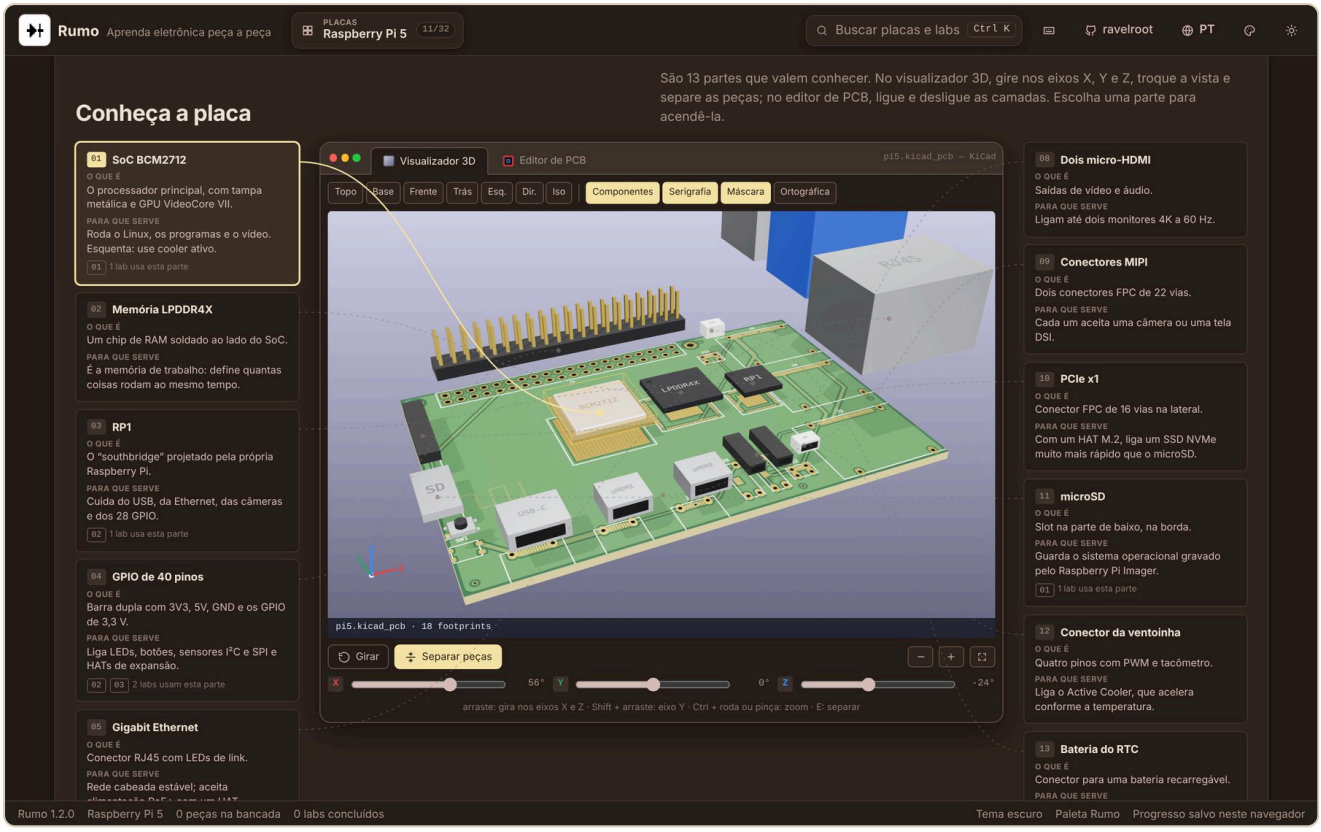
O visualizador 3D

A seção "Conheça a placa" tem três colunas: cartões das partes à esquerda e à direita e, no meio, uma janela no estilo do KiCad com duas abas: Visualizador 3D e Editor de PCB. O título da janela mostra o arquivo do projeto, por exemplo `esp32-s3.kicad_pcb`. Em telas estreitas a janela vai para cima e os cartões descem. Sem WebGL, a janela abre direto no editor de PCB, com um aviso.

O visualizador desenha a placa em WebGL com materiais físicos: FR-4 na borda, máscara de solda com o cobre aparecendo por baixo, pads em ouro (ENIG) ou estanho (HASL), serigrafia com as referências, furos, vias e cada componente modelado, sob luz de estúdio com reflexos e sombras suaves. O fundo em degradê e o indicador de eixos (X vermelho, Y verde, Z azul) seguem o visualizador do KiCad.

Controle	O que faz
Topo, Base, Frente, Trás, Esq., Dir., Iso	Vistas prontas, como os botões de vista do KiCad
Componentes, Serigrafia, Máscara	Mostram ou escondem os modelos 3D, a serigrafia e a máscara de solda (sem máscara, aparece o cobre nu)
Ortográfica	Troca a câmera em perspectiva pela ortográfica
Arrastar	Gira no eixo Z (como um prato) com o movimento horizontal e no eixo X (inclinação) com o vertical
Shift + arrastar	Gira no eixo Y com o movimento horizontal
Deslizadores X, Y e Z	Ajustam cada eixo de −180° a 180°, com o valor ao lado
Girar	Liga e desliga o giro automático no eixo Z
Separar peças	Vista explodida: cada componente sobe proporcionalmente à própria altura, com animação
− e +, Ctrl + roda, pinça	Zoom de 0,5× a 2,4×
Centralizar	Volta para a pose inicial: X 56°, Y 0°, Z −24°
Passar o mouse	A barra da janela mostra a referência, o valor e a biblioteca do footprint, por exemplo `U3 ATMEGA328P · Package_DIP:DIP-28_W7.62mm_Socket`
Teclado (com a vista em foco)	Setas giram X e Z, PgUp e PgDn giram Y, + e − fazem zoom, 0 centraliza, E separa e G gira

Cada cartão de parte traz o número, o nome, o que é, para que serve e os labs que usam aquela parte. Escolher um cartão, ou clicar na peça no 3D ou no editor, acende a peça com um brilho e uma caixa de seleção, destaca o cartão com a cor de destaque da paleta e acende a linha que liga os dois. Passar o mouse sobre um cartão também destaca a linha.



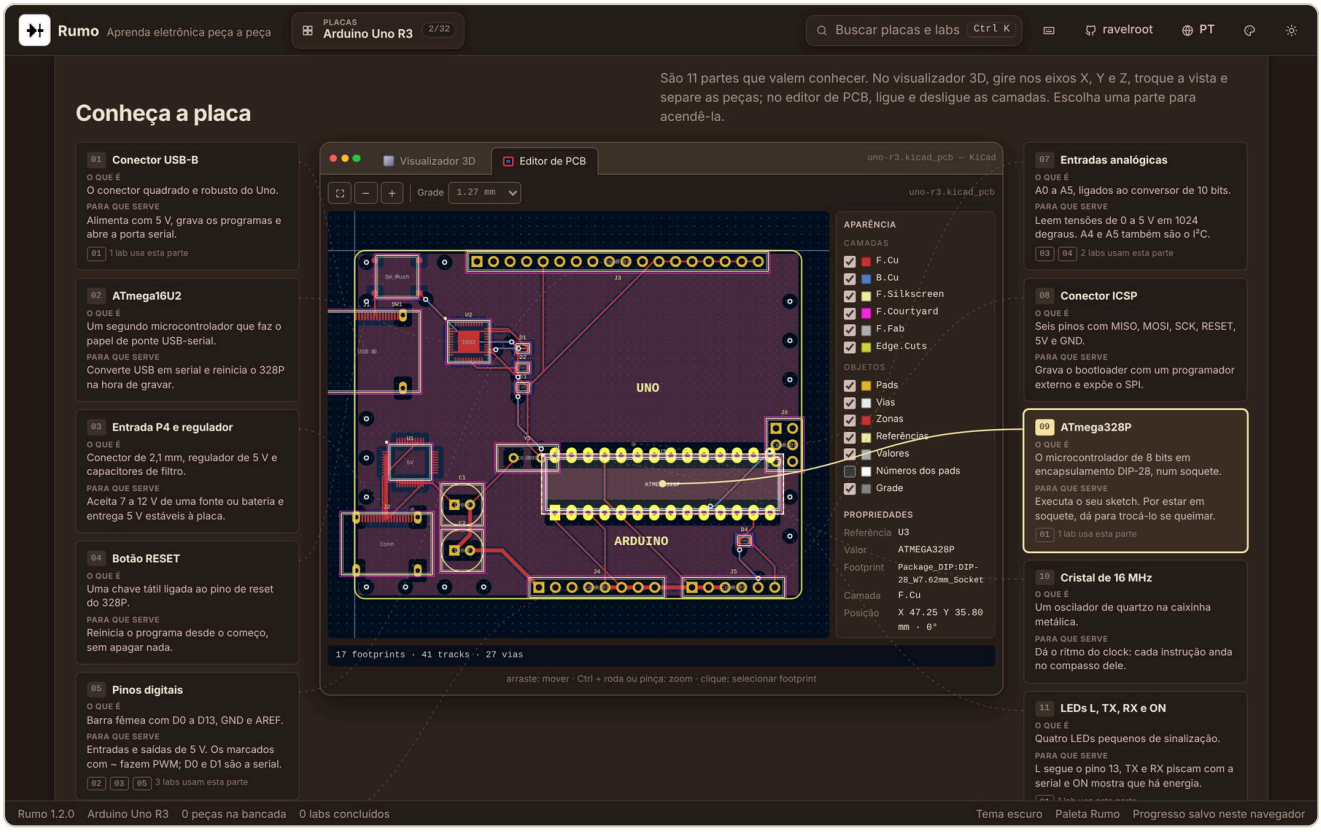
Raspberry Pi 5 na vista explodida, no tema escuro, com o SoC aceso.

O editor de PCB

A aba Editor de PCB mostra o mesmo projeto em 2D, com as cores do Pcbnew: fundo azul-escuro, F.Cu em vermelho, B.Cu em azul, F.Silkscreen em creme, F.Courtyard em magenta, F.Fab em cinza, Edge.Cuts em amarelo, pads THT dourados e vias claras. As zonas de cobre aparecem com o isolamento em volta de pads e vias, e as trilhas seguem 0°, 45° e 90°.

Controle	O que faz
Ajustar, − e +	Enquadra a placa, afasta e aproxima
Grade	De 0,635 a 50,8 mm, como no KiCad. A grade se adapta ao zoom para não virar ruído
Arrastar	Move a vista
Ctrl + roda, pinça	Zoom no ponto do cursor
Cursor	Cruz que salta para a grade; a barra de status da janela mostra X, Y, o zoom e a grade em milímetros
Clique num footprint	Seleciona o footprint, acende a parte e preenche as propriedades
Teclado (com o editor em foco)	Setas movem, + e − fazem zoom e 0 ou Home ajustam a placa

O painel Aparência liga e desliga cada camada (F.Cu, B.Cu, F.Silkscreen, F.Courtyard, F.Fab e Edge.Cuts) e cada objeto (pads, vias, zonas, referências, valores, números dos pads e grade). Os números dos pads só aparecem com zoom suficiente. O painel Propriedades mostra referência, valor, footprint, camada, posição, corpo, número de pads, montagem (SMD ou THT) e a parte da placa.



Arduino Uno R3 no editor de PCB, no tema escuro, com o ATmega328P selecionado.

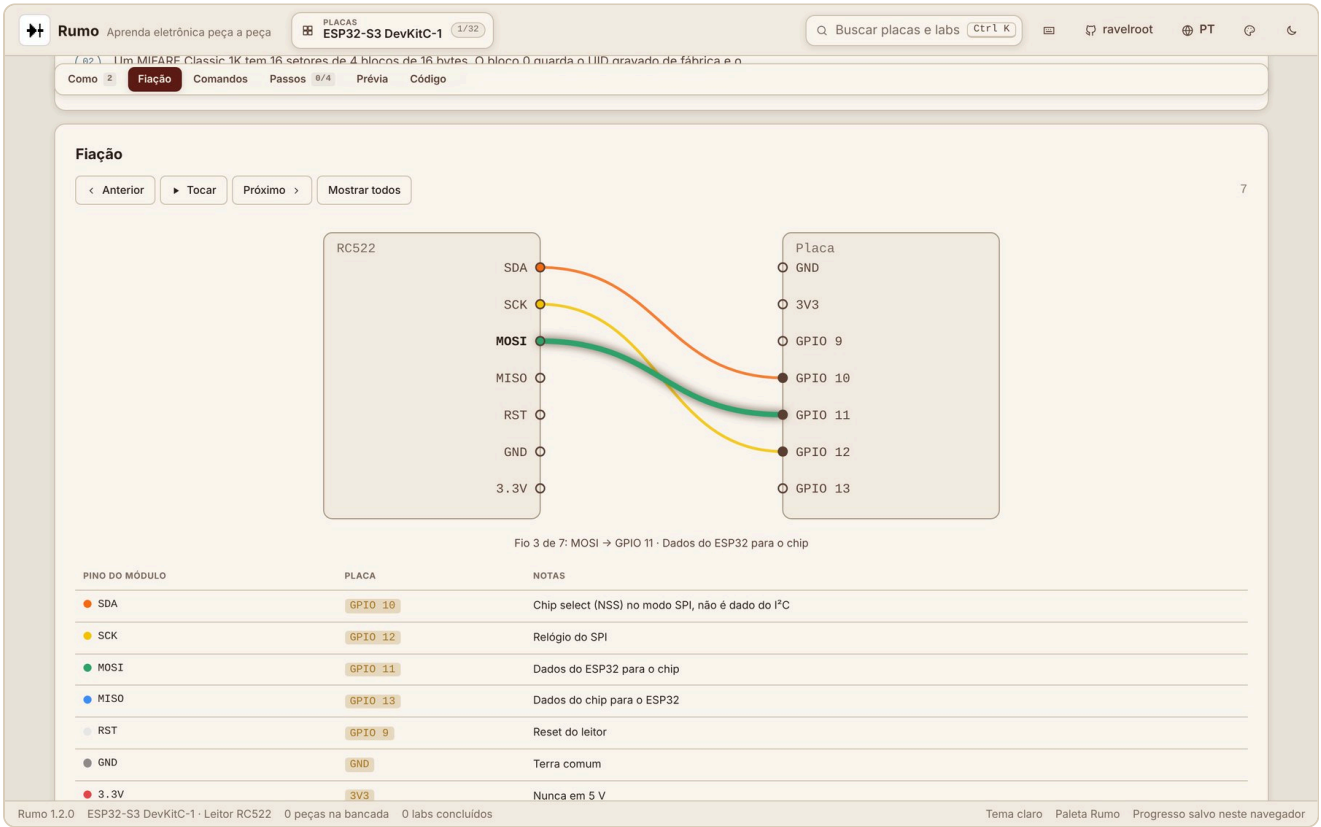
Os laboratórios

Um lab abre em `#/placa/lab`. O cabeçalho mostra a trilha (placa, tema e número), o título, o resumo, as peças (na cor oliva se você marcou na bancada, tracejadas se faltam, e um clique alterna) e as partes da placa envolvidas.

A barra de abas, num controle segmentado como no Pluma, fica fixa logo abaixo da barra superior e acompanha a rolagem:

Aba	Conteúdo
Como	A teoria do lab em blocos numerados
Fiação	Diagrama com o módulo à esquerda e os pinos da placa à direita. Anterior, Tocar, Próximo e Mostrar todos percorrem fio a fio, com o fio atual em destaque e a legenda "Fio 2 de 7: SCK → GPIO 12 · Relógio do SPI". A tabela abaixo lista pino do módulo, pino da placa e nota; um clique numa linha mostra aquele fio
Comandos	Cada comando com rótulo e botão Copiar. Os labs de ESP32 e Uno usam o `arduino-cli`, os de Pico usam o `mpremote` e os de Raspberry Pi usam o terminal
Passos	Checklist salvo no navegador. Ao marcar o último passo o lab vira "feito"; o botão "Marcar como feito" faz o mesmo à mão
Prévia	O simulador do lab, com uma linha explicando o que observar
Código	O arquivo completo com nome e botão Copiar

No rodapé, um cartão na cor de destaque da paleta leva ao próximo lab, avisando quando é o fim do tema, e há atalhos para o lab anterior, a placa e o topo.

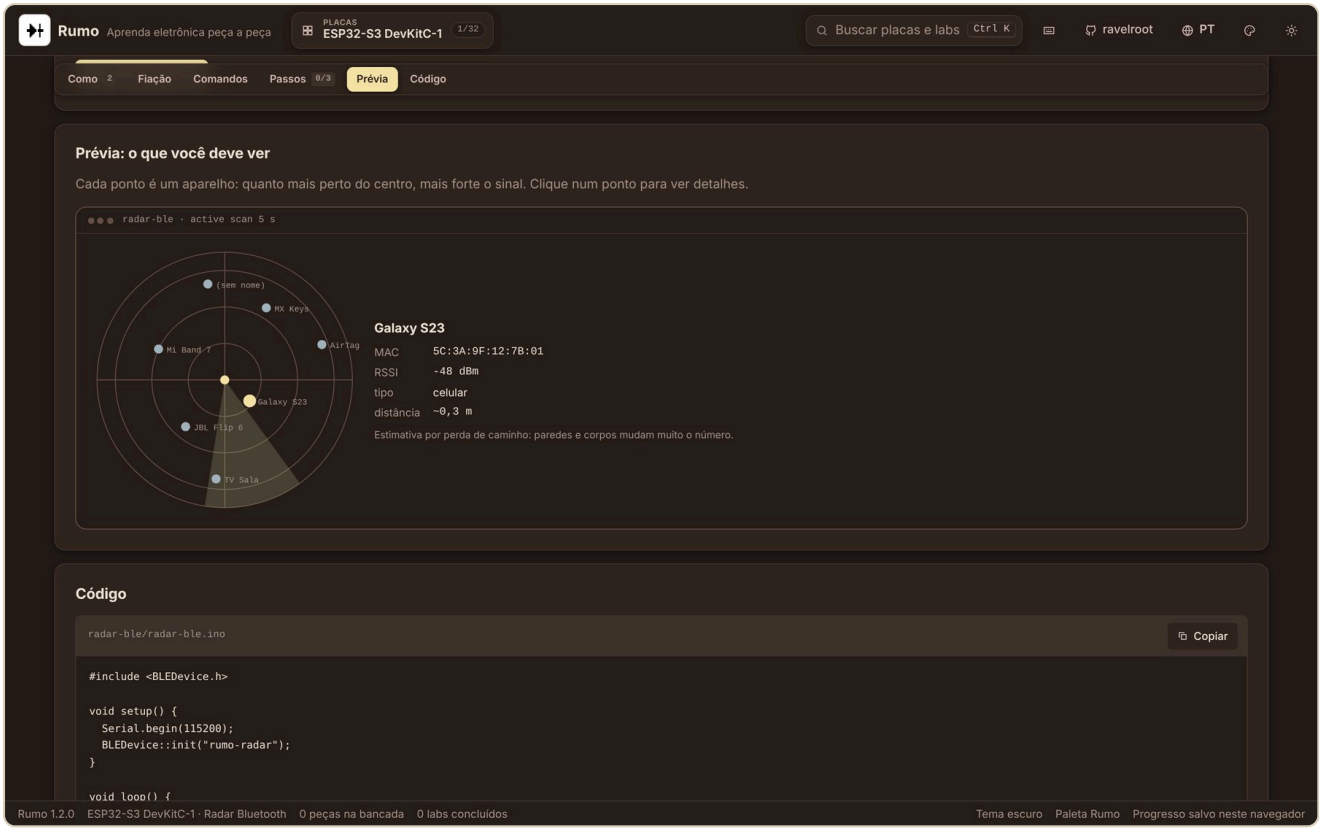


Fiação do leitor RC522 passo a passo.

Os 16 simuladores

Todos rodam no navegador, sem rede. Os que têm animação ou temporizador são desligados ao sair da página. As telas simuladas (monitor serial, displays, LEDs) mantêm as cores dos aparelhos reais; as abas e os seletores seguem a paleta.

Tipo	Nome	O que mostra
`serial`	Monitor serial	Linhas de boot com horário, campo de comando, botões com os comandos do sketch e, quando o lab tem, um LED que acende, pisca, respira ou faz SOS
`wifi`	Varredura de Wi-Fi	Lista de redes com barras de sinal, canais de 1 a 13 com a sobreposição de cada rede e o medidor de RSSI em dBm
`radar`	Radar Bluetooth	Aparelhos BLE posicionados pela intensidade, com varredura animada e detalhe de MAC, RSSI, tipo e distância estimada
`sintetizador`	Sintetizador	Teclado de uma oitava, quatro formas de onda, osciloscópio, espectro de harmônicos e som real pelo Web Audio, só depois de um toque
`microfone`	Dois microfones	Escopo do INMP441 e do KY-037 para palma, fala, assobio e silêncio, com as vistas de bias e de comparador
`rfid`	Leitor RFID	Cartão que se aproxima, UID, ATQA e SAK, a memória de 16 setores e um bloco de valor com créditos
`tela`	Tela	ST7789 colorida ou LCD 16×2, trocando de tela como o botão faria
`pwm`	PWM	Ciclo de trabalho de 0 a 255 com a onda, a tensão média e o brilho do LED
`servo`	Servo	Potenciômetro, braço do servo e largura do pulso a cada 20 ms
`distancia`	Ultrassom	Objeto a uma distância ajustável, pulso TRIG, largura do ECHO e o cálculo em centímetros
`pinagem`	Mapa dos 40 pinos	Conector do Raspberry Pi colorido por função, com filtro e detalhe do número BCM
`memoria`	Slots de memória	Quatro slots em dois canais: o modo resultante, a banda e a recomendação do manual
`post`	LEDs de POST	CPU, DRAM, VGA e BOOT acendendo em ordem, com cinco cenários
`timer555`	Temporizador 555	R1, R2 e C escolhidos em listas, carga do capacitor entre 1/3 e 2/3 de VCC, saída e LED
`regulador`	Regulador 7805	Entrada, carga e dissipador definem saída, calor, temperatura do chip e eficiência
`ponteh`	Ponte H	IN1, IN2 e PWM no ENA acendem as chaves certas, giram o motor e marcam a tabela verdade



Prévia do radar Bluetooth no tema escuro.

Arquitetura do código

```
src/
  app/        aparencia.ts, estado.ts (Loja), rotas.ts, progresso.ts, contexto.ts
  pcb/        modelo.ts, gerador.ts
  dados/      tipos.ts, base.ts, pecas.ts
    placas/       microcontroladores, computadores, pc, modulos, circuitos, index
    laboratorios/ esp32s3, outros, index
  i18n/       textos.ts
  ui/         dom.ts, icones.ts, cabecalho.ts, menu.ts, barra-status.ts, dialogos.ts,
              catalogo.ts, pagina-placa.ts, pagina-lab.ts, fiacao.ts
    kicad/        janela, visor3d, cena, modelos3d, texturas, editor2d, miniaturas
    simuladores/  base, serial, radio, audio, rfid, telas, sinais, pc, index
  css/        estilo.css
  main.ts     entrada, rotas e atalhos
public/       logo.svg e aparencia.js
```

Estado

A Loja guarda idioma, tema, paleta, rota, peças da bancada, passos marcados, labs feitos e as janelas abertas. `alterar()` só avisa os ouvintes quando algo muda de fato. Preferências vão para `rumo:preferencias` (idioma, tema e paleta) e o progresso para `rumo:progresso` no `localStorage`. Na leitura, `lerProgresso()` descarta ids que não existem no catálogo, duplicados, textos longos e passos fora do intervalo, e a paleta só é aceita se estiver na lista.

As páginas se inscrevem com `vigiar()`, que cancela a inscrição sozinha quando o elemento sai da tela. Cada página devolve `destruir()`, que para timers, animações, áudio e observadores.

Aparência

O módulo `src/app/aparencia.ts` define os tipos `Tema` e `Paleta`, a lista `PALETAS` (id, nome e as quatro cores do menu), o tema do sistema por `prefers-color-scheme` e `aplicarAparencia()`, que grava `data-tema` e `data-paleta` no `html` e ajusta o `color-scheme` e a cor `theme-color`. O `public/aparencia.js` faz o mesmo antes da primeira pintura, lendo `rumo:preferencias` e aceitando só valores conhecidos; por ser um arquivo do próprio site, passa pela política `script-src 'self'`. O menu de paletas (`src/ui/menu.ts`) abre ao lado do botão, navega pelas setas, fecha com `Esc` ou com um clique fora e pinta as amostras pelo CSSOM.

Rotas

`lerRota()` aceita `#/placas`, `#/placa` e `#/placa/lab`. Cada parte do endereço é reduzida a letras minúsculas, números e hífen, com até 48 caracteres. Placa desconhecida leva ao catálogo; lab de outra placa leva à placa.

Bancada

`situacao()` classifica um lab em feito, pronto, a uma peça ou faltam peças. `resumirBancada()` devolve prontos, a uma peça (com a peça), quantos pedem mais e quantos estão feitos. `pecasUsadas()` conta em quantos labs cada peça aparece.

O gerador de projetos de PCB

`projetoDe(placa)`, em `src/pcb/gerador.ts`, transforma o desenho de uma placa num projeto de PCB e guarda o resultado em memória. O gerador é determinístico: uma semente tirada do id da placa alimenta um gerador pseudoaleatório, então a mesma placa gera sempre o mesmo projeto. No total são 410 footprints, mais de 12 mil pads, 1.020 trilhas e 677 vias.

Etapa	O que faz
Classificar	`classificar()` escolhe um de 38 modelos (QFN, QFP, SOIC, SOT, DIP, BGA, blindagem, conector, barra de pinos, slot, eletrolítico, MLCC, cristal, dissipador, soquete LGA e outros) pela forma, tamanho, altura e rótulo
Referência e biblioteca	Prefixo do KiCad (U, J, SW, D, C, R, Y, HS, BT, RV, L, FAN, T, DS, LS, W, AE, K) com numeração por placa, e um nome no padrão `Biblioteca:Footprint`, como `Package_QFP:LQFP-64_10x10mm_P0.5mm`
Pads	Pads SMD e THT com formato retangular, redondo, oval ou arredondado, no passo do encapsulamento; matrizes para BGA e LGA; contatos dourados de borda, bordas castelladas e pads fora da placa descartados
Furos e serigrafia	Furos de fixação nos cantos, contorno de serigrafia, ponto do pino 1, pátio (courtyard), contorno de fabricação e posição da referência
Trilhas e vias	Liga pads de footprints vizinhos com rotas octilineares; parte vai por F.Cu e parte desce por vias para B.Cu. As trilhas de energia são mais largas, e vias de costura correm pelas bordas
Zonas	Plano de terra nas duas faces, com isolamento em volta de pads, vias e trilhas

Como funciona o renderizador 3D

O visualizador usa three.js com `WebGLRenderer`, tone mapping ACES, sombras PCF suaves e um ambiente de estúdio (`RoomEnvironment` com PMREM, criado uma vez por renderizador). A placa é uma caixa com a espessura real e seis materiais: FR-4 nas laterais e as faces de cima e de baixo desenhadas em canvas por `desenharFace()`, com máscara, cobre sob a máscara, trilhas, pads com acabamento, vias, furos e serigrafia. Um segundo canvas, de `desenharPbr()`, vira o mapa de metal e rugosidade, para que só o cobre exposto reflita a luz.

Cada footprint ganha um modelo em `modelos3d.ts`, montado com caixas, caixas arredondadas, cilindros, cápsulas e `InstancedMesh` para pernas, pinos e aletas. Os materiais (epóxi, plástico, metal, alumínio, ouro, estanho, cerâmica, vidro e borracha) são compartilhados e descartados juntos. A câmera fica fixa e o grupo da placa gira pela pose X, Y e Z; a imagem só é renderizada quando algo muda.

A seleção clona o material da peça com emissão e desenha uma caixa envolvente. O clique usa `Raycaster`. As linhas entre cartões e peças são um SVG por cima da seção, desenhado na cor de destaque da paleta: a cada movimento o visualizador projeta os cantos de cada parte na tela, e o editor mede os retângulos dos footprints. Ao sair da página, cena, texturas, geometrias e o contexto WebGL são liberados.

O editor de PCB é um SVG com um grupo por camada e por tipo de objeto, o que deixa ligar e desligar cada um com uma linha. As zonas usam máscaras SVG para o isolamento, e o zoom e o movimento alteram só o `viewBox`.

Formato de dados

Placa

```
export const EXEMPLO: Placa = {
  id: 'exemplo',
  nome: 'Placa de exemplo',
  sigla: 'EXEMPLO',
  fabricante: t('Genérico', 'Generic'),
  categoria: 'modulo',
  resumo: t('Uma frase.', 'One sentence.'),
  usos: t('Onde se usa.', 'Where it is used.'),
  essenciais: t('a placa e um cabo', 'the board and a cable'),
  especificacoes: [espec(R.tensao, '5 V'), ...],
  desenho: {
    largura: 40, altura: 20, cor: 'azul',
    componentes: [
      c('chip', 15, 6, 8, 8, 1.2, 'mcu', 'MCU'),
      c('barra', 2, 17, 25.4, 2.54, 2.6, 'pinos'),
    ],
  },
  partes: [
    parte('mcu', t('Microcontrolador', 'Microcontroller'),
      t('O que é.', 'What it is.'), t('Para que serve.', 'What it does.')),
    parte('pinos', ...),
  ],
  temas: [],
};
```

`c(forma, x, y, largura, altura, z, parte?, rótulo?, cor?)` usa milímetros a partir do canto de cima à esquerda. Componentes sem parte são só decoração. Toda parte precisa aparecer em pelo menos um componente, e os testes conferem.

Laboratório

```
{
  id: 'meu-lab', placa: 'exemplo', tema: 'inicio', numero: 1, icone: 'led',
  titulo: t(...), resumo: t(...),
  pecas: ['protoboard'], partes: ['mcu'],
  como: [t(...), t(...)],
  fiacao: { modulo: 'Sensor', ligacoes: [fio('SDA', 'GPIO 8', 'verde', t(...))] },
  comandos: [{ rotulo: t(...), linha: 'arduino-cli compile ...' }],
  passos: [t(...), t(...), t(...)],
  previa: { tipo: 'serial', inicio: ['...'], comandos: [{ nome: 'on', resposta: ['...'] }] },
  explicacaoPrevia: t(...),
  codigo: { arquivo: 'meu-lab/meu-lab.ino', texto: `...` },
}
```

Como adicionar

Uma placa

1. Escolha o arquivo da categoria em `src/dados/placas/` e crie a constante da placa.
2. Meça a placa em milímetros (largura e altura) e posicione os componentes com `c()`. Comece pelos maiores e use o nome da parte em cada um que pertence a ela.
3. Escreva as partes com `parte()`: nome, o que é e para que serve, nas duas línguas.
4. Adicione a constante à lista `PLACAS` em `src/dados/placas/index.ts`, na posição em que deve aparecer.
5. Rode `npm test`: os testes acusam parte sem desenho, componente fora da placa ou texto vazio.
6. Rode `npm run dev`, abra `#/id-da-placa`, gire, separe as peças e confira as linhas.

Um laboratório

1. Garanta que a placa tem o tema em `temas`.
2. Se o lab usa uma peça nova, crie-a em `src/dados/pecas.ts`.
3. Crie o objeto do lab em `src/dados/laboratorios/` com número único na placa.
4. Escolha a prévia entre os 16 tipos. Para um monitor serial, escreva as linhas iniciais e as respostas de cada comando.
5. Rode os testes: eles conferem placa, tema, partes, peças, fiação, passos e que o código não tem comentários.

Testes e segurança

Testes e CI

Unitários (Vitest, 329 testes): pelo menos 30 placas, ids únicos e as cinco categorias; para cada placa, textos nas duas línguas, quatro ou mais especificações, partes com o que é e para que serve, toda parte desenhada e todo componente dentro da placa; para cada projeto de PCB, semente determinística, cor da máscara, referências únicas no padrão do KiCad, bibliotecas válidas, toda parte com footprint, pads, matrizes, trilhas, vias e furos dentro da placa e trilhas só a 0°, 45° ou 90°; para cada lab, referências válidas, fiação completa, passos, comandos seriais únicos e código sem comentários; a bancada, a leitura segura do progresso e as rotas.

Interface (Playwright, 11 testes): página inicial com o visualizador 3D, giro nos três eixos e pelo teclado, vista explodida, acender parte no 3D e no editor de PCB, camadas do editor, catálogo com 32 placas, bancada e passos salvos após recarregar, monitor serial com LED e erro, fiação fio a fio, busca, idioma e tema pelo teclado a partir do tema do sistema, menu e atalho de paletas com a escolha salva, e logo, título e barra de status.

```
npm test
npx playwright install chromium
npm run test:e2e
```

Segurança

- CSP no build: `default-src 'self'`, `script-src 'self'`, `style-src 'self'`, sem objetos, sem formulários e com `upgrade-insecure-requests`.
- Nenhum `innerHTML`, `eval` ou atributo `style` montado a partir de texto: estilos dinâmicos, como as amostras do menu de paletas, usam o CSSOM, permitido pela CSP.
- Leitura do `localStorage` validada contra o catálogo, com limite de tamanho; tema e paleta só aceitam valores conhecidos, no `aparencia.js` e na loja.
- Rotas higienizadas e sem uso do endereço para montar HTML.
- Links externos com `rel="noopener noreferrer"`.
- Sem servidor, cookies ou rastreadores; o áudio só começa com interação.
- A única dependência de produção é o three.js, com versão fixa; a interface usa as fontes do sistema. `npm audit --omit=dev` no CI e Dependabot semanal.
- Os labs de placa-mãe, fonte ATX e fonte linear envolvem energia elétrica. O Rumo indica onde há tensão perigosa, mas não substitui o manual do fabricante nem cuidados de segurança na bancada.

Personalização

Paletas de cores

As quatro paletas seguem as do Grafo, do Crivo e do Pluma. Cada uma tem um bloco para o tema claro e outro para o escuro em `src/css/estilo.css`, escolhidos pelos atributos `data-paleta` e `data-tema` do elemento `html`.

Paleta	Origem	Tema claro	Tema escuro
Rumo (padrão)	paleta da casa, a mesma do Grafo, do Crivo e do Pluma	linho `#E6DDCE`, vinho `#5D1C16`, ardósia `#A3B4BE`, palha `#F6E4A6`	fundo `#1D1613`, superfície `#30251F`, destaque palha `#F6E4A6`
Bento	painel administrativo	névoa `#F5F4F4`, cinza `#BAB7B6`, grafite `#4D4B4B`, tinta `#000000`	fundo `#020202`, superfície `#1E1D1D`, destaque branco
Sonda	motor de busca	areia `#F3F1EC`, azul `#7FA7B8`, noite `#17212B`, magenta `#A8235A`	fundo `#0F161B`, superfície `#151E25`, destaque rosa `#F0679A`
Nexo	paleta original do Nexo	papel `#E8E8E8`, pó `#D9D1CE`, malva `#AB9D9D`, tinta `#000000`	fundo `#000000`, superfície `#221E1D`, destaque papel `#E8E8E8`

Para criar uma paleta, acrescente os dois blocos em `estilo.css`, inclua a paleta em `PALETAS` (`src/app/aparencia.ts`) e o nome na lista aceita por `public/aparencia.js`.

Tokens, fontes e cores do KiCad

Os tokens da paleta (`--bg`, `--surface`, `--surface-2`, `--text`, `--border`, `--accent`, `--accent-soft`, `--highlight` e as cores `--c-oliva`, `--c-ouro`, `--c-anil`, `--c-brasa`) ficam no começo de `src/css/estilo.css`; os componentes usam nomes próprios (`--mesa`, `--caixa`, `--texto`, `--tampa`) que apontam para eles. As fontes são as do sistema: `--font` (`system-ui`) e `--mono` (`ui-monospace`).

As cores de máscara ficam em `COR_MASCARA` (`src/pcb/gerador.ts`), as cores e materiais 3D em `texturas.ts` e `modelos3d.ts` e as cores das camadas do editor nas classes `.k-*` do CSS; elas imitam o KiCad e não mudam com a paleta. Os textos da interface ficam em `src/i18n/textos.ts`; os textos das placas e labs, nos próprios dados. A marca (nome, versão, repositório e grade da logo) fica em `src/marca.ts`, e a logo é desenhada em preto num quadrado branco na barra superior, no ícone do site e nas imagens do README.

Solução de problemas

Sintoma	Causa provável e solução
Página em branco ao abrir o `index.html` direto	Abra pelo `npm run preview` ou pelo GitHub Pages; o navegador bloqueia módulos em `file://`
Pages publica 404	Em Settings → Pages, a fonte precisa ser GitHub Actions
A aba Visualizador 3D aparece desativada	O navegador não ofereceu WebGL. Ative a aceleração gráfica ou use Chrome, Edge, Firefox ou Safari atuais; o editor de PCB continua funcionando
O 3D fica lento	Placas grandes, como a ATX, têm centenas de peças. Feche outras abas com 3D ou desligue Componentes na barra
Sem som no sintetizador	Toque numa tecla (o navegador exige interação) e confira o volume da aba
Peças e passos sumiram	Foram limpos os dados do site ou é outra janela anônima; o progresso fica só neste navegador
O tema abre escuro (ou claro) sem ter sido escolhido	Na primeira visita o tema segue o do sistema. Escolha pelo botão ou pela tecla `T`; a escolha fica salva
As cores parecem antigas depois de atualizar	O navegador guardou o `index.html` anterior. Recarregue com `Ctrl` + `F5`
`npm ci` falha	Confira o Node 22 (`node -v`) e se o `package-lock.json` está no commit
Playwright não acha o navegador	Rode `npx playwright install chromium` ou aponte `RUMO_CHROME` para um Chrome instalado

Atalhos e licença

Atalhos de teclado

Tecla	Ação
`Ctrl` + `K` ou `/`	Buscar placas, partes e labs
`P`	Catálogo
`,` e `.`	Placa anterior e próxima
`[` e `]`	Lab anterior e próximo
Setas, PgUp, PgDn, `+`, `−`, `0`	Visualizador 3D em foco: girar, zoom e centralizar
Setas, `+`, `−`, `0`, Home	Editor de PCB em foco: mover, zoom e ajustar
`E` e `G`	Separar peças e giro automático
`T`	Tema claro ou escuro
`C`	Próxima paleta de cores NOVO
`L`	Português ou inglês
`Esc` e `?`	Fechar janela ou menu e folha de atalhos

Licença

Copyright (©) 2026 Erik Pimenta. Todos os direitos reservados. O Rumo usa uma licença de uso própria, no arquivo `LICENSE`, em português e inglês, que permite apenas usar e baixar.

Permitido	Não permitido sem autorização por escrito
Usar a versão publicada, de graça, para fins pessoais, educacionais ou profissionais	Modificar, adaptar, traduzir ou criar obras derivadas
Baixar o repositório ou uma versão de Releases e rodar no próprio computador, exatamente como foi publicado	Vender, alugar, licenciar, sublicenciar, cobrar pelo acesso ou incluir em produto ou serviço pago
	Redistribuir, publicar, espelhar ou hospedar em outro endereço, mesmo sem alterações
	Reutilizar o código, os textos, os dados das placas e dos labs, os desenhos, os modelos 3D, as animações, o nome ou a logo
	Remover ou alterar os avisos de direitos ou a licença

Cada cópia baixada é só para quem baixou e deve manter o arquivo `LICENSE`. Os forks dentro do GitHub, garantidos pelos Termos de Serviço da plataforma, não dão direito de modificar, publicar ou vender. O three.js e as ferramentas de

terceiros continuam sob as próprias licenças. Uso fora da licença encerra as permissões, e as cópias baixadas devem ser apagadas. Pedidos de autorização: pelo perfil ravelroot no GitHub.

Catálogo completo

As 32 placas e circuitos

Placa	Categoria	Partes identificadas	Labs
ESP32-S3 DevKitC-1 `#/esp32-s3`	Microcontroladores	Portas USB-C, Ponte USB-serial, Botões BOOT e RST, Regulador de 3,3 V, LED RGB, Módulo ESP32-S3, Antena de PCB, Barras de pinos	9
Arduino Uno R3 `#/uno-r3`	Microcontroladores	Conector USB-B, ATmega16U2, Entrada P4 e regulador, Botão RESET, Pinos digitais, Pinos de energia, Entradas analógicas, Conector ICSP, ATmega328P, Cristal de 16 MHz, LEDs L, TX, RX e ON	5
Raspberry Pi Pico `#/pico`	Microcontroladores	Micro-USB, Botão BOOTSEL, LED da placa, Flash QSPI, RP2040, Cristal de 12 MHz, Conversor de energia, Bordas castelladas, Pontos SWD	2
Arduino Nano `#/nano`	Microcontroladores	Mini-USB, CH340, ATmega328P e cristal, Botão RESET, Regulador AMS1117, LEDs de sinal, Barras de pinos, ICSP	—
Arduino Mega 2560 `#/mega2560`	Microcontroladores	USB-B, ATmega16U2, Entrada e regulador, RESET, Pinos digitais D0–D21, Pinos de energia, A0–A15, Barra dupla D22–D53, ICSP, ATmega2560	—
NodeMCU ESP8266 v3 `#/nodemcu`	Microcontroladores	Micro-USB, Ponte USB-serial, FLASH e RST, Regulador de 3,3 V, Módulo ESP-12E, Antena de PCB, LED azul, Pinos D0–D8, A0 e energia	—
ESP32-CAM `#/esp32-cam`	Microcontroladores	ESP32, Antena e conector IPEX, Câmera OV2640, LED de flash, PSRAM de 4 MB, Slot microSD, RESET, Pinos	—
STM32 Blue Pill `#/bluepill`	Microcontroladores	Micro-USB, Pinos PA, PB e PC, Conector SWD, Jumpers BOOT0 e BOOT1, STM32F103C8T6, Cristais, RESET, LEDs, Regulador	—
Seeed XIAO ESP32-C3 `#/xiao-c3`	Microcontroladores	USB-C, ESP32-C3, Conector de antena, BOOT e RESET, LED de carga, Pinos	—
BBC micro:bit v2 `#/microbit`	Microcontroladores	Matriz de LEDs 5×5, Botões A e B, Logo sensível ao toque, Micro-USB, nRF52833, Alto-falante, Microfone MEMS, Conector de borda	—
Raspberry Pi 5 `#/pi5`	Computadores de placa única	SoC BCM2712, Memória LPDDR4X, RP1, GPIO de 40 pinos, Gigabit Ethernet, USB 2.0 e USB 3.0, USB-C e botão liga, Dois micro-HDMI, Conectores MIPI, PCIe x1, microSD, Conector da ventoinha, Bateria do RTC	3
Raspberry Pi Zero 2 W `#/pi-zero-2w`	Computadores de placa única	RP3A0, GPIO sem barra, Antena, Mini-HDMI, Micro-USB OTG, Micro-USB PWR, microSD, Conector CSI, LED de atividade	—
Placa-mãe ATX `#/atx`	Hardware de PC	Soquete do processador, VRM e dissipadores, Slots de memória DDR5, Conector ATX de 24 pinos, Conector EPS de 8 pinos, Painel traseiro de E/S, Slots PCI Express, Slots M.2, Chipset, Portas SATA, BIOS e bateria CR2032, Painel frontal, Conectores USB internos, Conectores de ventoinha, LEDs de diagnóstico, Áudio	3

Placa	Categoria	Partes identificadas	Labs
Placa-mãe Mini-ITX `#/mini-itx`	Hardware de PC	Soquete LGA1700, VRM, Dois slots DDR5, ATX 24 pinos, EPS 8 pinos, Painel traseiro, Placa Wi-Fi M.2, Slot M.2, Chipset, SATA, PCIe x16	—
Placa de vídeo (PCB) `#/gpu`	Hardware de PC	Chip gráfico, Memória GDDR6, VRM, Conector PCIe de 8 pinos, Contatos PCIe x16, Saídas de vídeo, VBIOS, Conector das ventoinhas	—
SSD M.2 NVMe 2280 `#/ssd-nvme`	Hardware de PC	Conector chave M, Controlador, Cache DRAM, Flash NAND, PMIC, Meia-lua do parafuso	—
Memória DDR5 UDIMM `#/ddr5`	Hardware de PC	Chips de DRAM, PMIC, SPD, Capacitores de desacoplamento, Contatos e entalhe	—
Fonte ATX (vista interna) `#/fonte-atx`	Hardware de PC	Entrada AC e chave, Filtro de EMI, Ponte retificadora, Capacitor principal, Lado primário, Transformador, Lado secundário, Filtro de saída, Controle e supervisão, Cabos de saída	—
Ponte H L298N `#/l298n`	Módulos	L298N e dissipador, Saídas dos motores, Diodos de roda livre, Capacitores, Regulador 78M05 e jumper, Borne de alimentação, Entradas de controle, LED	1
Módulo relé 1 canal 5 V `#/rele`	Módulos	Pinos de entrada, Optoacoplador, Transistor, Diodo de proteção, Relé, LEDs, Bornes NA, COM e NF	—
Sensor ultrassônico HC-SR04 `#/hcsr04`	Módulos	Transdutor emissor (T), Transdutor receptor (R), Circuito de controle, VCC, TRIG, ECHO e GND	—
Leitor RFID RC522 `#/rc522`	Módulos	Antena em espira, MFRC522, Cristal de 27,12 MHz, Filtro de EMC, LED, Pinos SPI	—
Driver de passo A4988 `#/a4988`	Módulos	Chip A4988, Dissipador, Trimpot de corrente, Resistores de medição, Pinos	—
Regulador buck LM2596 `#/lm2596`	Módulos	Entrada IN+ e IN−, Saída OUT+ e OUT−, LM2596, Indutor de 33 µH, Capacitores, Trimpot multivoltas, Diodo Schottky	—
Carregador de lítio TP4056 `#/tp4056`	Módulos	USB-C, TP4056, Resistor de programação, DW01A e FS8205A, LEDs, B+ e B−, OUT+ e OUT−	—
Acelerômetro e giroscópio MPU-6050 `#/mpu6050`	Módulos	MPU-6050, Regulador de 3,3 V, Resistores de pull-up, LED, Pinos	—
Tela OLED 0,96" SSD1306 `#/oled-ssd1306`	Módulos	Painel OLED, SSD1306, Cabo flexível, GND, VCC, SCL e SDA, Resistor de endereço	—

Placa	Categoria	Partes identificadas	Labs
GPS u-blox NEO-6M `#/neo6m`	Módulos	NEO-6M, Conector e antena, Bateria de backup, EEPROM, Regulador, LED de PPS, VCC, RX, TX e GND	—
Pisca-pisca com NE555 `#/ne555`	Circuitos	NE555, R1 e R2, Trimpot, Capacitores de tempo e controle, LED e resistor, Borne de entrada	1
Fonte regulada com LM7805 `#/lm7805`	Circuitos	LM7805 e dissipador, Entrada e capacitor de 470 µF, Saída e capacitor de 100 µF, Capacitores de 100 nF, Diodo 1N4007, LED indicador	1
Amplificador de áudio LM386 `#/lm386`	Circuitos	LM386, Entrada P2, Potenciômetro de volume, Capacitor de ganho, Capacitor de saída, Rede Zobel, Saída do alto-falante, Alimentação	—
Fonte linear com ponte de diodos `#/retificador`	Circuitos	Fusível, Entrada AC, Transformador, Ponte de diodos, Capacitor de filtro, Resistor de sangria, LED, Saída DC	—

Os 25 laboratórios

Lab	Placa	Tema	Fiação	Prévia	Código
Olá, bancada `#/esp32-s3/ola`	ESP32-S3 DevKitC-1	Início	—	Monitor serial	sim
LED RGB `#/esp32-s3/led-rgb`	ESP32-S3 DevKitC-1	Início	—	Monitor serial	sim
Botão BOOT `#/esp32-s3/botao-boot`	ESP32-S3 DevKitC-1	Início	—	Monitor serial	sim
Tela ST7789 `#/esp32-s3/tela-st7789`	ESP32-S3 DevKitC-1	Tela e entrada	sim	Tela	sim
Varredura de Wi-Fi `#/esp32-s3/wifi-scan`	ESP32-S3 DevKitC-1	Wi-Fi	—	Varredura de Wi-Fi	sim
Leitor RC522 `#/esp32-s3/rfid-rc522`	ESP32-S3 DevKitC-1	RFID e NFC	sim	Leitor RFID	sim
Dois microfones `#/esp32-s3/microfones`	ESP32-S3 DevKitC-1	Sensores	sim	Escopo de microfones	sim
Radar Bluetooth `#/esp32-s3/radar-ble`	ESP32-S3 DevKitC-1	Bluetooth LE	—	Radar BLE	sim
Sintetizador I²S `#/esp32-s3/sintetizador`	ESP32-S3 DevKitC-1	Som	sim	Sintetizador	sim
Pisca o L `#/uno-r3/uno-blink`	Arduino Uno R3	Início	—	Monitor serial	sim

Lab	Placa	Tema	Fiação	Prévia	Código
Brilho com PWM `#/uno-r3/uno-fade`	Arduino Uno R3	PWM e movimento	sim	PWM	sim
Servo com potenciômetro `#/uno-r3/uno-servo`	Arduino Uno R3	PWM e movimento	sim	Servo	sim
LCD 16×2 por I²C `#/uno-r3/uno-lcd`	Arduino Uno R3	Tela	sim	Tela	sim
Distância por ultrassom `#/uno-r3/uno-ultrassom`	Arduino Uno R3	Sensores	sim	Ultrassom	sim
MicroPython e LED `#/pico/pico-blink`	Raspberry Pi Pico	Início	—	Monitor serial	sim
Termômetro interno `#/pico/pico-temperatura`	Raspberry Pi Pico	Sensores	—	Monitor serial	sim
Gravar o sistema `#/pi5/pi-sistema`	Raspberry Pi 5	Sistema	—	Monitor serial	—
LED e botão com gpiozero `#/pi5/pi-gpio`	Raspberry Pi 5	GPIO	sim	Monitor serial	sim
Mapa dos 40 pinos `#/pi5/pi-pinagem`	Raspberry Pi 5	GPIO	—	Mapa de pinos	—
Processador e cooler `#/atx/atx-cpu`	Placa-mãe ATX	Montagem	—	LEDs de POST	—
Memória em dual channel `#/atx/atx-memoria`	Placa-mãe ATX	Montagem	—	Slots de memória	—
Primeira partida na bancada `#/atx/atx-bancada`	Placa-mãe ATX	Primeira partida	—	LEDs de POST	—
Ajustar o pisca `#/ne555/ne555-pisca`	Pisca-pisca com NE555	Circuito	—	Temporizador 555	—
Quanto esquenta o 7805 `#/lm7805/lm7805-fonte`	Fonte regulada com LM7805	Circuito	—	Regulador linear	—
Sentido e velocidade `#/l298n/l298n-motor`	Ponte H L298N	Circuito	sim	Ponte H	sim